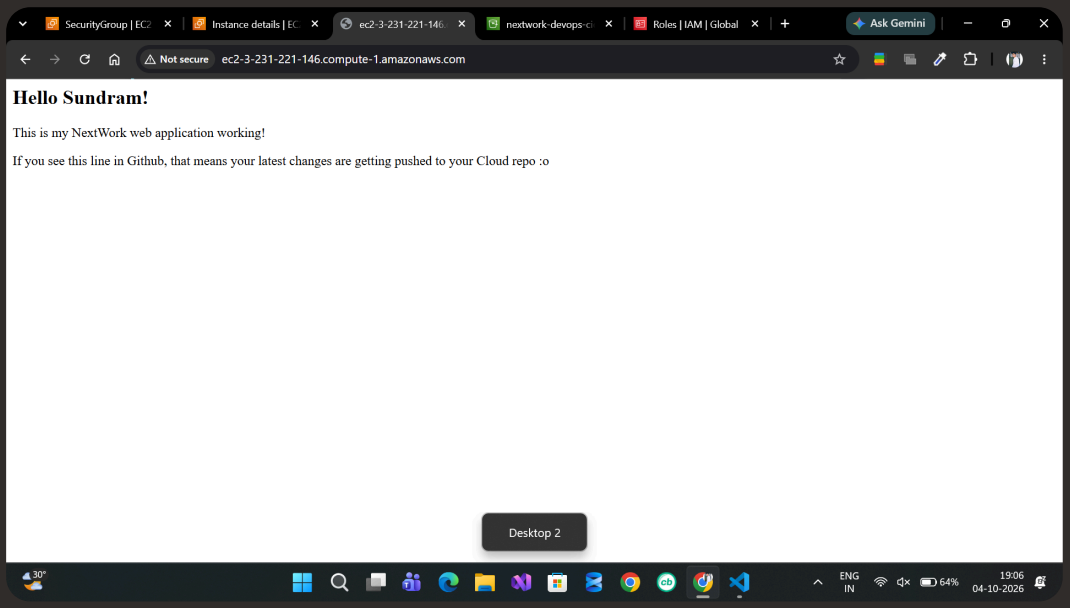
DEPLOY A WEB APP WITH CODEDEPLOY

By Sundram Singhal | October 2026



Introducing Today's Project!

In this project, I will demonstrate how to automate the deployment I'm doing this project to learn how automatic deployment works, how it handle the errors and rollback accordingly and allow us not to copy files.

Key tools and concepts

Services I used were AWS EC2, AWS S3, AWS CodeArtifact, AWS CloudFormation, AWS CodeDeploy, AWS IAM, and GitHub. Key concepts I learnt include CI/CD pipeline automation, Infrastructure as Code (IaC) with CloudFormation templates, building & packaging Java web artifacts using Maven, managing CodeDeploy lifecycle events with appspec.yml, shell scripting for deployment automation, and scoping security with IAM roles and Security Groups.

Project reflection

This project took me approximately 3 hours to complete. The most challenging part was setting up the codebuild part as in my region the limit was 0 then i build by project from the vs code terminal itself. It was most rewarding to complete the project and i feeled extremly satisfied when i saw that my project was deployed successfully.

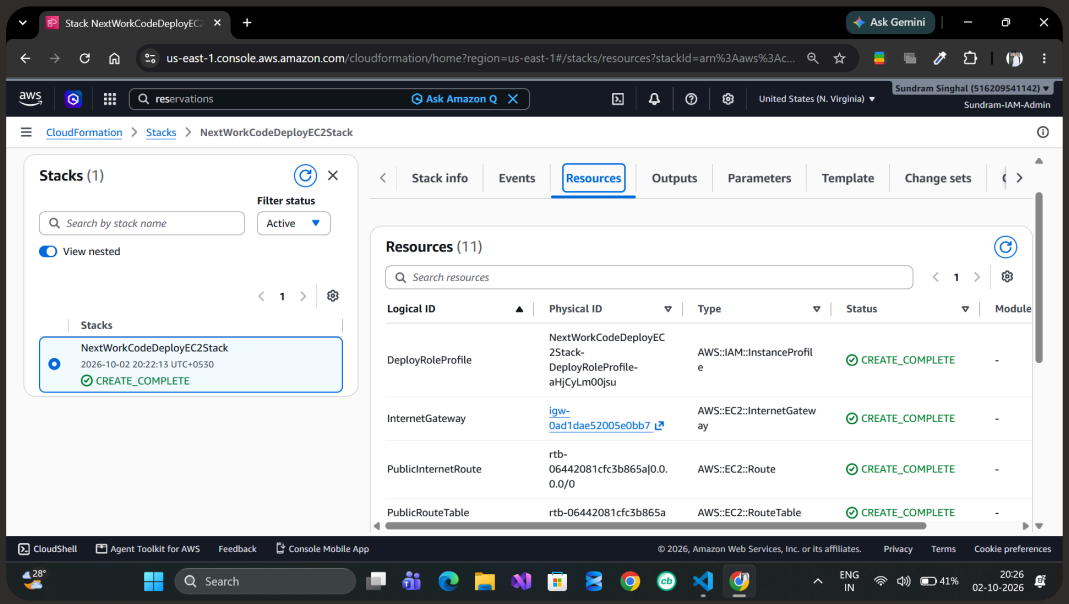
This project is part five of a series of DevOps projects where I'm building a CI/CD pipeline! I'll be working on the next project from tomorrow itself.

Deployment Environment

To set up for CodeDeploy, I launched an EC2 instance and VPC because the EC2 instance acts as the web server to host and run our Java web application, while the VPC provides a secure, isolated network for it.

Instead of launching these resources manually, I used AWS CloudFormation to automatically create and configure all infrastructure using a template file.
When I need to delete these resources, I can simply delete the CloudFormation stack, which will automatically remove all created resources at once.

Other resources created in this template include Security Groups, Subnets, an Internet Gateway, and an IAM Instance Profile (Role). They're also in the template because they provide internet connectivity, set up network routing, define firewall security rules to allow traffic from my IP, and grant the EC2 instance permissions to talk to CodeDeploy and S3.



Deployment Scripts

Scripts are like mini-programs that automate tasks. They're essentially text files filled with commands that you'd normally type one by one in the terminal, but packaged together so they run automatically in sequence. In our project, we're using scripts to automate deployment tasks.

The install_dependencies.sh script sets up all the software needed to run our website by installing programs (called Tomcat and Apache) that handle internet traffic and host our application. It then creates special settings that let these programs to work together, making our website accessible to visitors on the internet.

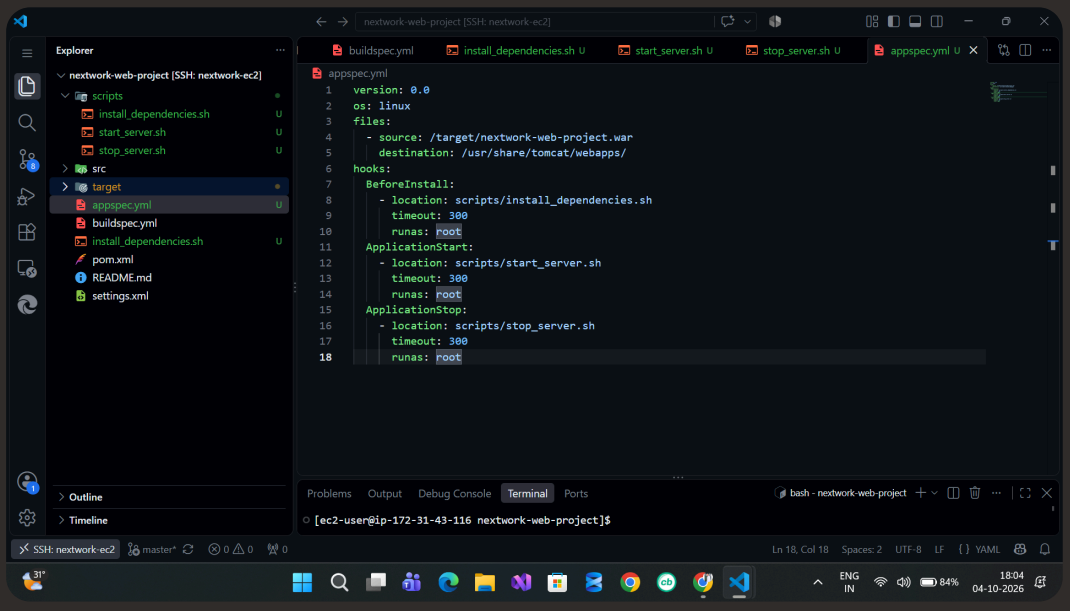
The start_server.sh starts both Tomcat (our Java application server) and Apache (our web server) and makes sure they'll restart automatically if the EC2 instance ever reboots.

The stop_server.sh will safely stops web server services by first checking if they're running. It uses pgrep to check for running processes of Apache (httpd) and Tomcat, and only attempts to stop the services if they are actually active. This prevents errors that could occur from trying to stop services that aren't running.

appspec.yml

Then, I wrote an appspec.yml file to provide instruction manual for CodeDeploy. The key sections in appspec.yml are version, os, files, hooks.
We can assume appspec.yml file as the choreographer that ensures everyone moves in the right sequence during deployment.

I also updated buildspec.yml because we are telling CodeBuild: "After you build the app, make sure to include these additional files in the final package." We're adding our appspec.yml and scripts folder because CodeDeploy will need them to properly deploy the application. Without them, CodeDeploy wouldn't know what to do with our compiled code!



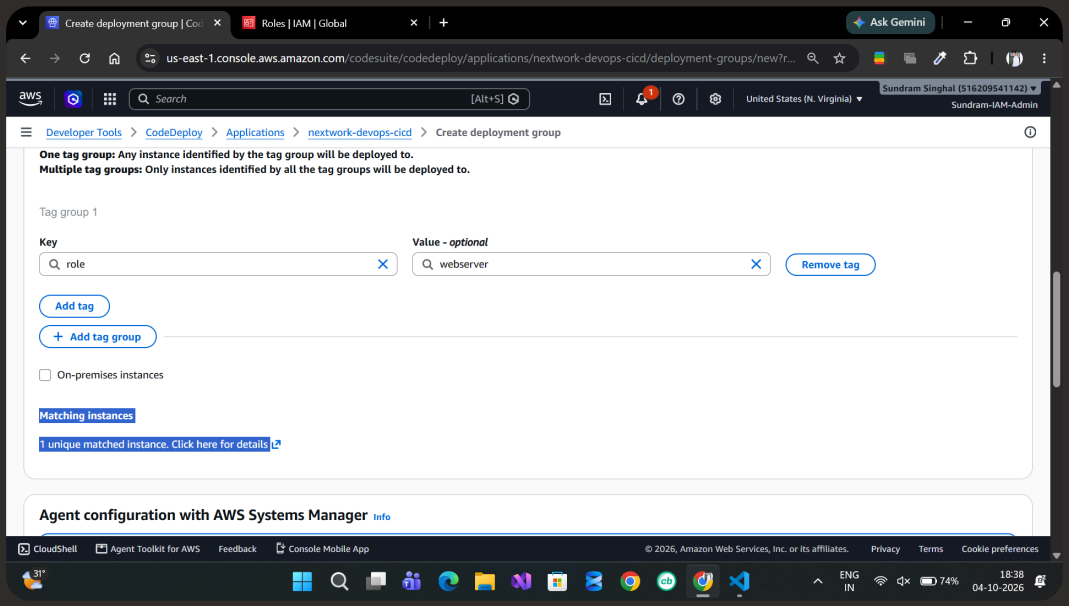
Setting Up CodeDeploy

A deployment group is a collection of EC2 instances that are grouped to deploy something together and A CodeDeploy application is like the main folder for your deployment project. It doesn't do much on its own, but it helps you organize everything related to deploying one application.

CodeDeploy needs IAM roles to get permissions to access and manage AWS resources on your behalf. These permissions let CodeDeploy do things like:

- Accessing EC2 instances to deploy applications.
- Reading application artifacts from S3 buckets.
- Updating Auto Scaling groups.
- Write CloudWatch logs about what it's doing

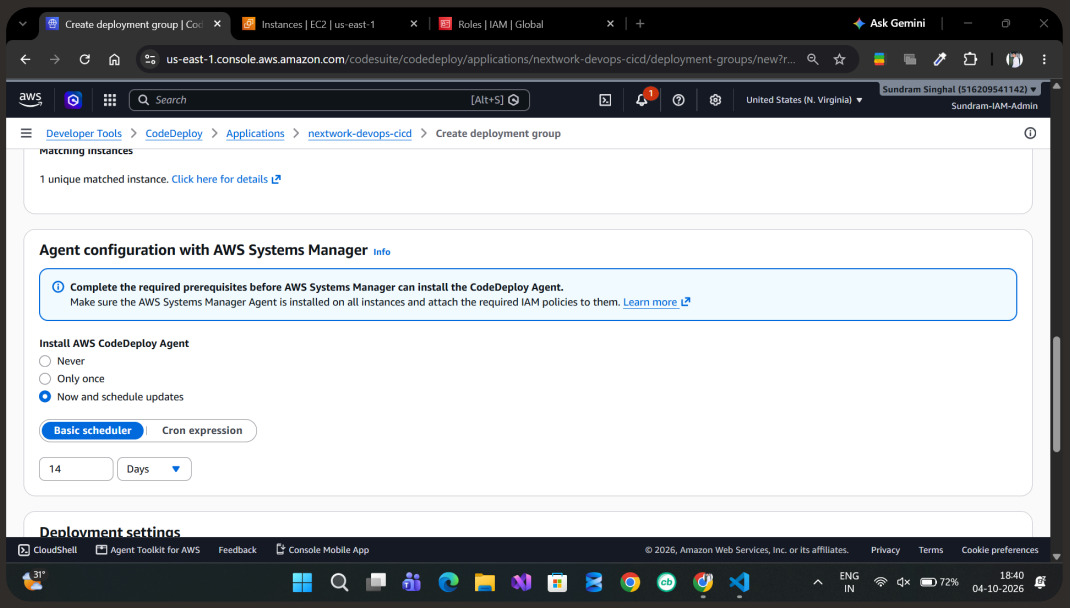
We are using Tags here so that CodeDeploy can identify the target EC2 instances for deployment. In our CloudFormation template, we tagged our EC2 instance with role: webserver. CodeDeploy will use this tag to find and deploy to the correct instance.



Deployment configurations

Another key settings is the deployment configuration, which affects how quickly you'd like to roll out your application. I used CodeDeployDefault.AllAtOnce because It's the fastest option, but also the riskiest - if something goes wrong, all my instances could be affected at once.

The CodeDeploy Agent is a software that lets your EC2 instance communicate with CodeDeploy.
Whenever you initiate a deployment, it's the CodeDeploy Agent that receives the deployment instructions from CodeDeploy and carries them out on the EC2 instance.

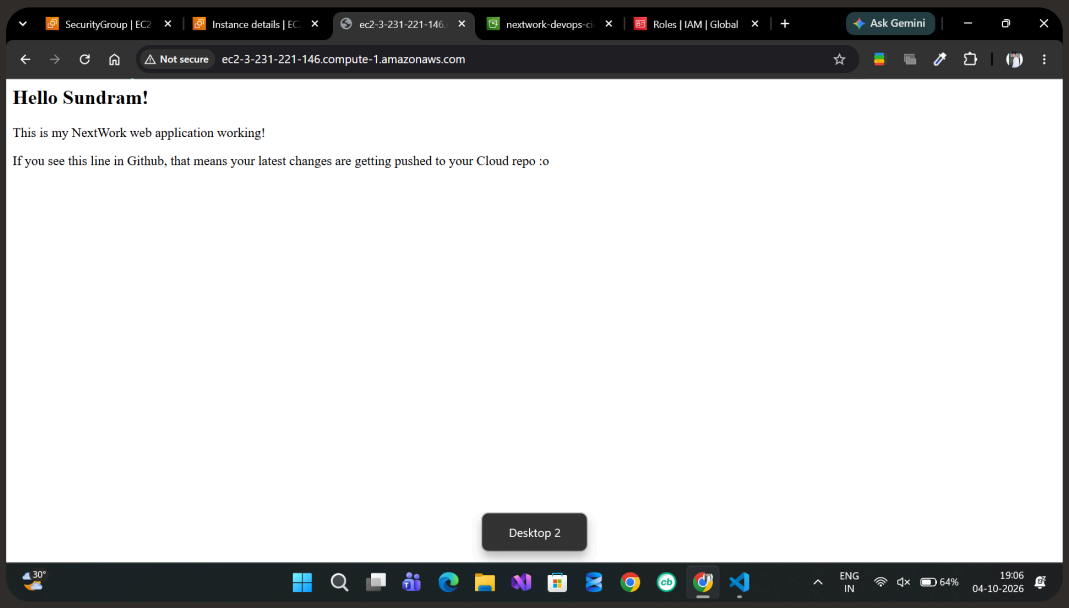


Success!

A CodeDeploy deployment represents a single update to your application, with its own unique ID and history. When you create a deployment, you're telling CodeDeploy that Which version of the application to deploy, Where to deploy it and How to deploy it.

The revision location is the place where CodeDeploy looks to find your application's build artifacts. We're using the S3 bucket that's storing our WAR file, so CodeDeploy knows where to find the latest version of our web app it's deploying to the deployment EC2 instances!

To check that the deployment was a success in event sections all events lists get succeeded then I selected the Instance ID in the Deployment lifecycle events panel. This takes me to the deployment EC2 instance I launched with CloudFormation and then I clicked on Public IPv4 DNS and i took 2 minutes to open my delpoyed project.



Disaster Recovery

In a project extension, I decided to make wrong chnges in my stop_server.sh file. The intentional error I created was i misspelled command that caused script to exit with non-zero status and the deployment got failed.

I also enabled rollbacks with this deployment, which means when w eenable rollback it head back our application to its previous state itself rather the spending so much time in finding errors in the deployment i.e rollbacks are super important because they help minimize downtime when things breaked.

When my deployment failed, the automatic rollback was not performed because it uses the latest code artifact To actually recover from the isuue while In production environments, true automated rollbacks are often implemented with more advanced tools. AWS CodePipeline can be configured to automatically roll back to the last successful deployment when a failure is detected, which eliminates the need for manual intervention.

